unpsock C++ Socket 库详细设计文档

1. 概述

1.1 项目定位

`unpsock` 是一个面向 Linux（POSIX 接口）的现代 C++ Socket 库。与 Asio 等跨平台异步库不同，`unpsock` 刻意放弃跨平台抽象与复杂异步模型，聚焦于 **SoC / 嵌入式 Linux 平台** 的易用性与性能：

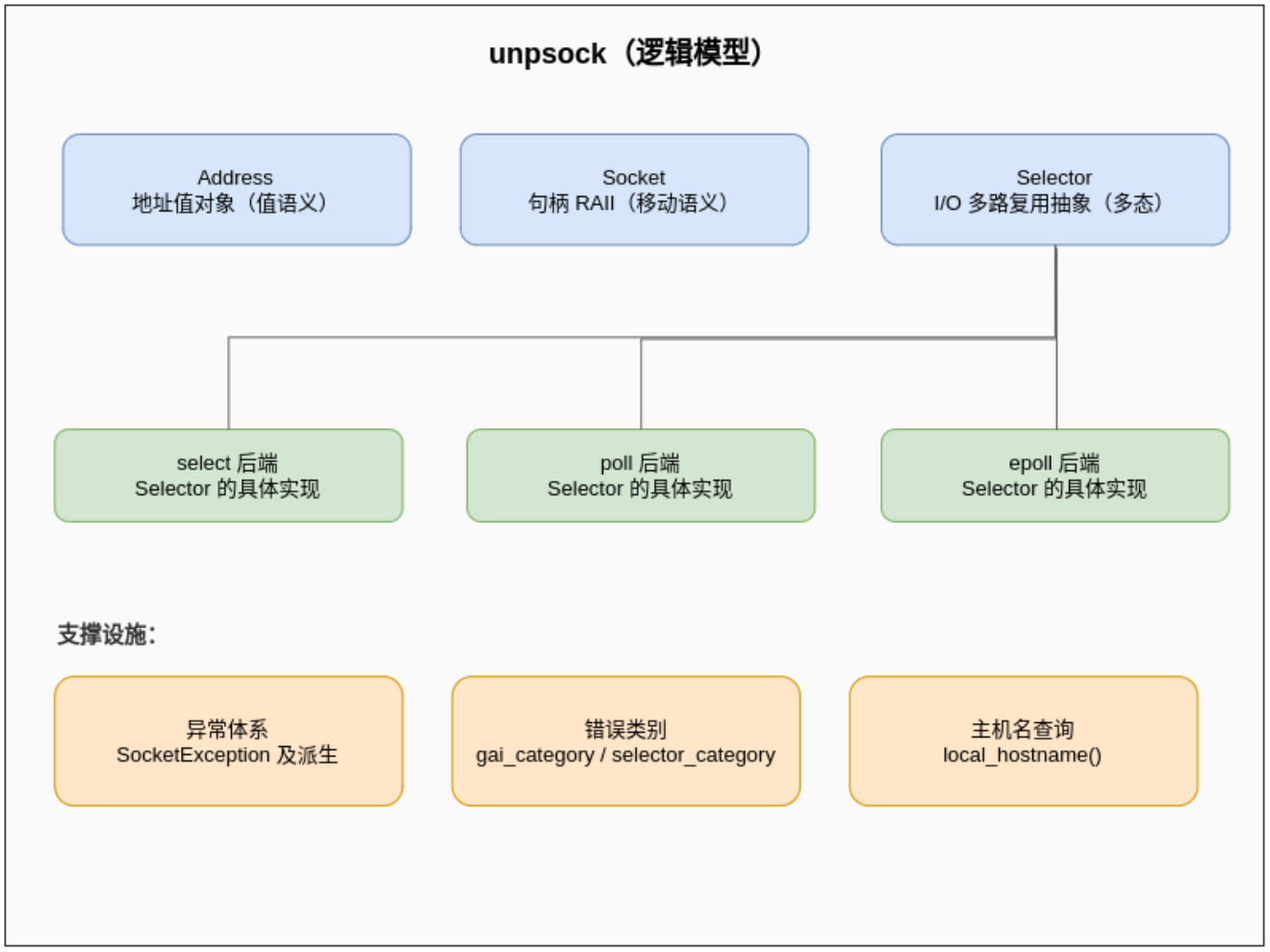
维度	unpsock 的选择	与 Asio 的区别
平台	仅 POSIX / Linux	不做 Windows / IOCP 抽象
并发模型	同步阻塞 + 可选 non-blocking + Selector	无 Proactor/Reactor 框架、无协程、无回调链
类层次	扁平（Address / Socket / Selector 三实体）	无模板策略、无 policy-based 设计
内存管理	`std::string` / `std::vector` 自动 RAII	调用方无需管理 buffer 生命周期
错误处理	**双风格**：`std::error_code` 与异常	与 Asio 类似的 `error_code` 版 + 自定义异常版并存
接口风格	参考 Python `socket` 模块	直白、命令式，学习成本低

1.2 设计目标与原则

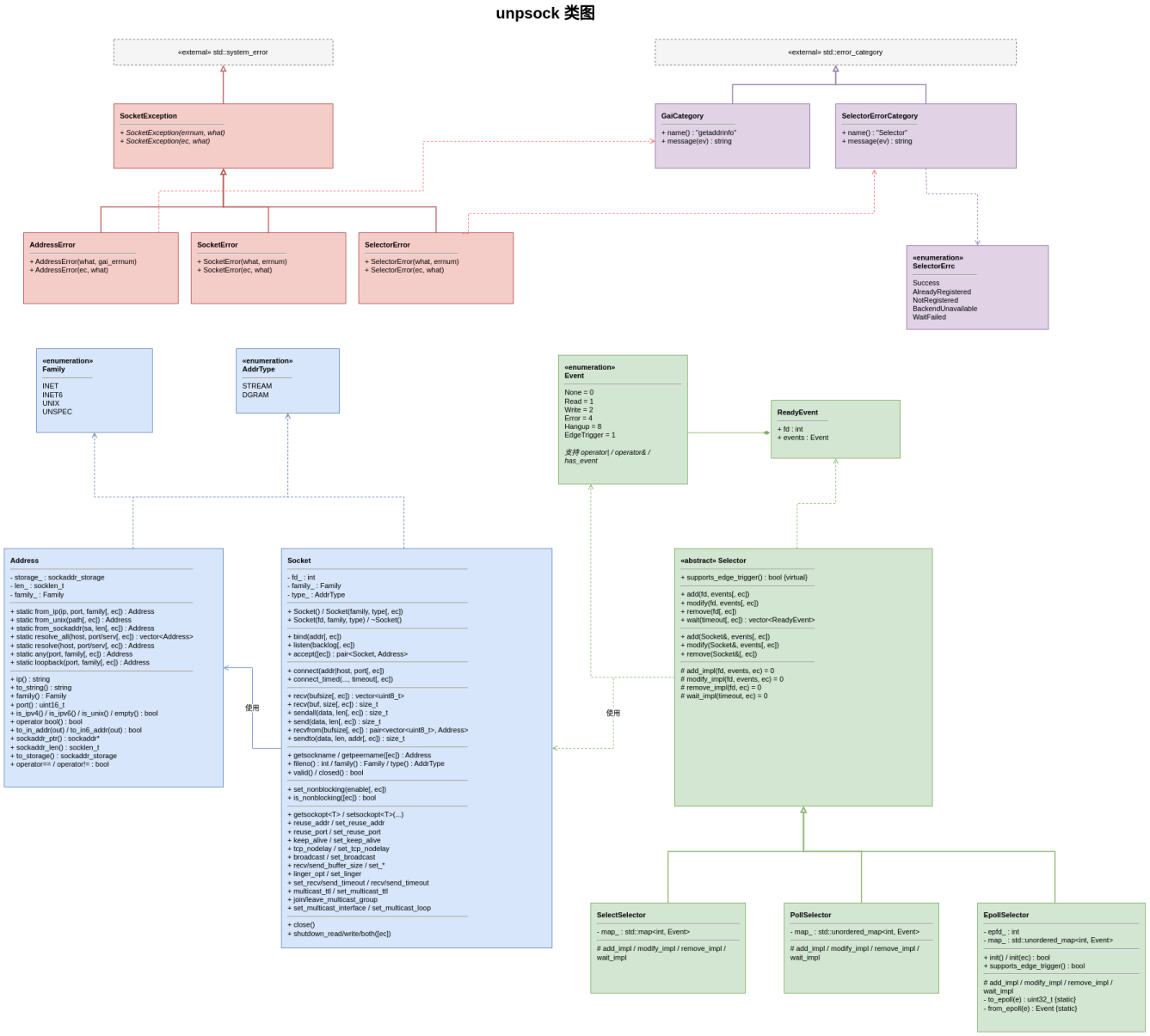
原则	说明
零隐藏成本	所有封装在非热点路径上；保留裸 fd、原始 `sockaddr` 指针等接口，允许调用方绕过封装直接使用 POSIX 接口
双错误风格	每个可能失败的接口同时提供 **异常版**（无 `ec` 参数，失败抛 `SocketError` / `AddressError` / `SelectorError`）和 **错误码版**（带 `std::error_code&` 出参，`noexcept`）。原型验证用前者，生产复杂逻辑用后者
POD 化的 kernel 参数	`getsockopt` / `setsockopt` 采用模板 + `socklen_t` 长度校验，避免 `int` / `timeval` / `linger` 等类型错配
不做过度抽象	`Selector` 只有四个虚函数（注册、修改、移除、等待），不做 "handler 注册"、不做 "事件循环"
RAII 完备	`Socket` 移动语义、析构自动关闭；`Address` 值语义；`Selector` 析构释放后端资源
header-only 交付	全部实现以 `inline` / 模板形式交付，零编译依赖，便于集成到嵌入式工程

1.3 逻辑模块组成

库在逻辑上由三个实体类和三个支撑设施构成：

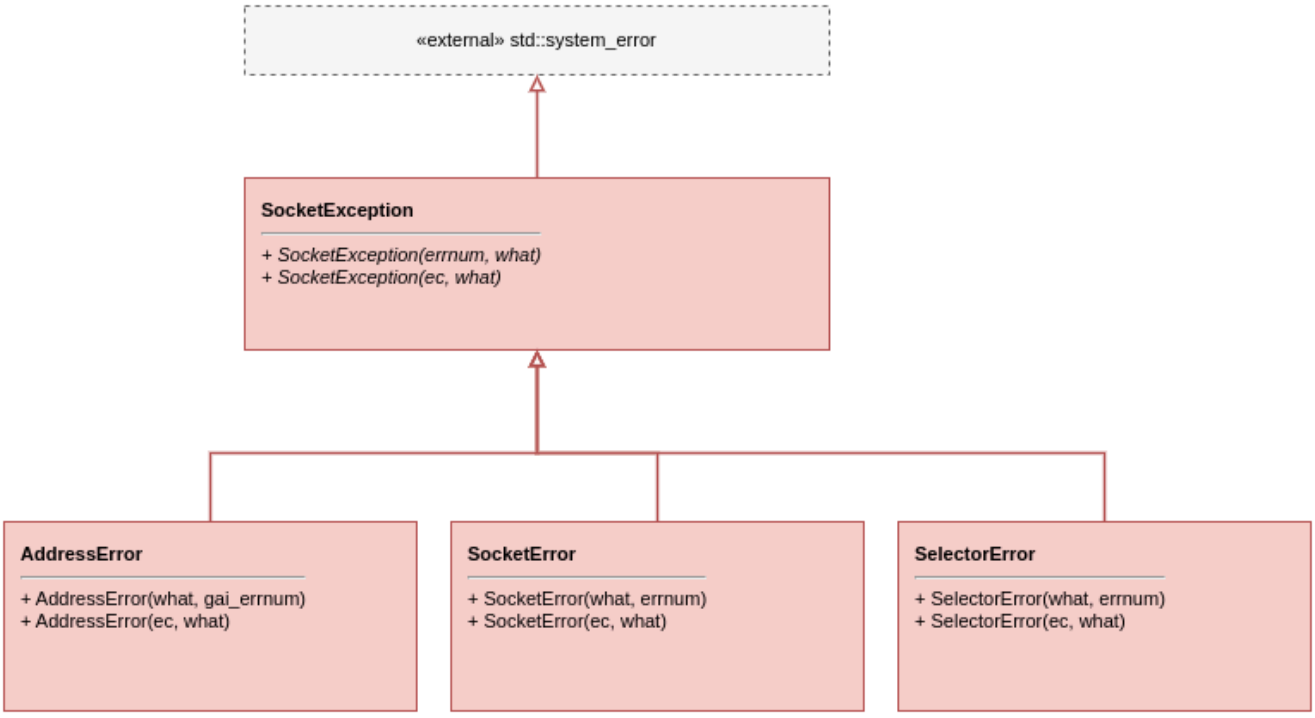


1.4 整体类关系



2. 错误处理设计

2.1 异常层次




```
std::system_error
  └── unpsock::SocketException          // 通用基类；持 std::error_code + what
        ├── unpsock::AddressError       // 地址解析 / DNS（默认 gai_category）
        ├── unpsock::SocketError        // 底层 socket syscall 失败
        └── unpsock::SelectorError      // Selector 注册 / wait 失败
```

关键点：

- `SocketException` 继承 `std::system_error`，构造函数同时接受 `int errnum` 和 `std::error_code`。
- `AddressError` 默认用 `gai_category()`，因为 `getaddrinfo` 的返回值属于 `EAI_*` 命名空间，不能映射到 `errno`。
- `SelectorError` 同时覆盖两种来源：`system_category`（后端初始化失败）与 `selector_category`（`AlreadyRegistered` 等）。

`SocketException` 契约：

```
class SocketException : public std::system_error {
    SocketException(int errnum, const std::string& what);
    SocketException(std::error_code ec, const std::string& what);
};
```

继承 `std::system_error` 以复用 `what()` 与 `code()`。

2.2 派生异常构造约定

AddressError：

```
class AddressError : public SocketException {
    explicit AddressError(const std::string& what, int gai_errnum = 0);
    explicit AddressError(std::error_code ec, const std::string& what);
};
```

- `(what, gai_errnum)` 重载默认将 `gai_errnum` 包装为 `make_gai_error_code(gai_errnum)`，适用于 `getaddrinfo` 风格错误；
- `(ec, what)` 重载原样透传错误码，适用于内部已持有完整 `error_code` 的场景（如 `from_sockaddr_impl` 产生的 `std::errc`）。

SocketError：

```
class SocketError : public SocketException {
    explicit SocketError(const std::string& what, int errnum = 0);
    explicit SocketError(std::error_code ec, const std::string& what);
};
```

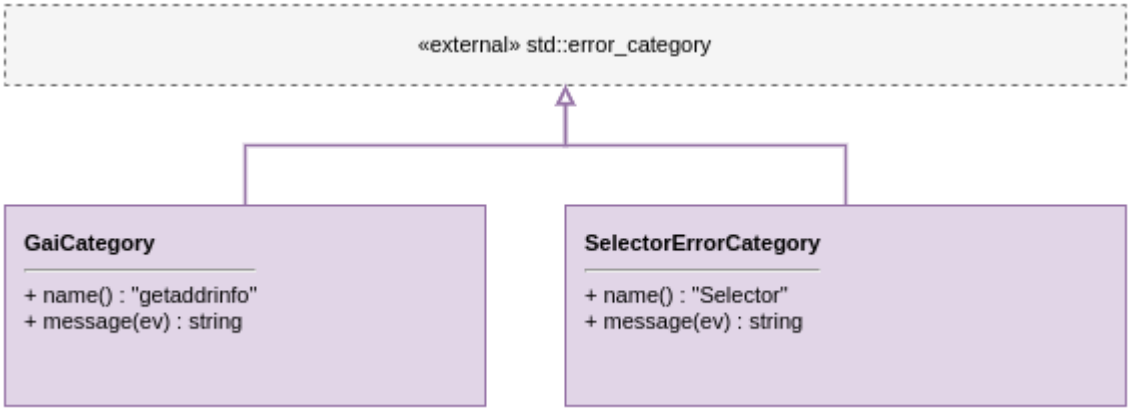
- `(what, errnum)` 重载将 `errnum` 包装为 `system_category`（默认 0 表示无系统错误）；
- `(ec, what)` 重载原样透传。注意：`connect(host, port)` 解析失败时 `ec` 携带的是 `gai_category`，通过此重载可完整保留。

SelectorError（定义于 Selector 模块）：

```
class SelectorError : public SocketException {
    explicit SelectorError(const std::string& what, int errnum = 0);
    explicit SelectorError(std::error_code ec, const std::string& what);
};
```

同时覆盖 `system_category`（后端初始化/等待失败）与 `selector_category`（注册状态错误）。

2.3 自定义 error_category



GaiCategory：承载 `getaddrinfo` 返回值。

```cpp
class GaiCategory : public std::error_category {
    const char* name() const noexcept override;         // "getaddrinfo"
    std::string message(int ev) const override;         // 委托 ::gai_strerror
};
inline const std::error_category& gai_category() noexcept;   // 单例
inline std::error_code make_gai_error_code(int ev) noexcept;
```

SelectorErrorCategory：承载 Selector 内部错误。

```cpp
enum class SelectorErrc {
    Success = 0,
    AlreadyRegistered,   // add 时 fd 已注册
    NotRegistered,       // modify/remove 时 fd 未注册
    BackendUnavailable,  // make_selector 找不到可用后端
    WaitFailed,          // wait 底层失败（errno 用 system_category）
};
```

`SelectorErrorCategory` 实现 `name()`（返回 `"Selector"`）/ `message()`（返回英文短语，便于日志检索），`make_selector_ec` 构造 `error_code`。

2.4 双风格约定（契约）

所有公有 API 遵循以下模式：

```cpp
// 异常版：无 error_code 出参，失败 throw
void op(Args...);
// 错误码版：error_code& 出参，noexcept，失败置 ec
void op(Args..., std::error_code& ec) noexcept;
```

约定细则：

1. 错误码版**总是先** `ec.clear()`，成功时 `ec` 为空。
2. 异常版内部调用错误码版，再 `if (ec) throw`——这保证两条路径行为一致。
3. 返回类型为「值」的错误码版，失败时返回**默认构造值**（如 `Address{}`、空 `vector`、0）。
4. 异常版的 `what` 消息中应包含必要的上下文（如地址串 `addr.to_string()`），便于定位。

2.5 错误类别划分

错误源	`error_category`	构造方式
系统调用 errno	`std::system_category()`	`std::error_code(errno, std::system_category())`
`std::errc` 通用语义	`std::generic_category()`	`std::make_error_code(std::errc::xxx)`
`getaddrinfo` 返回值	自定义 `gai_category`	`make_gai_error_code(rc)`
Selector 内部错误	自定义 `selector_category`	`make_selector_ec(SelectorErrc::xxx)`

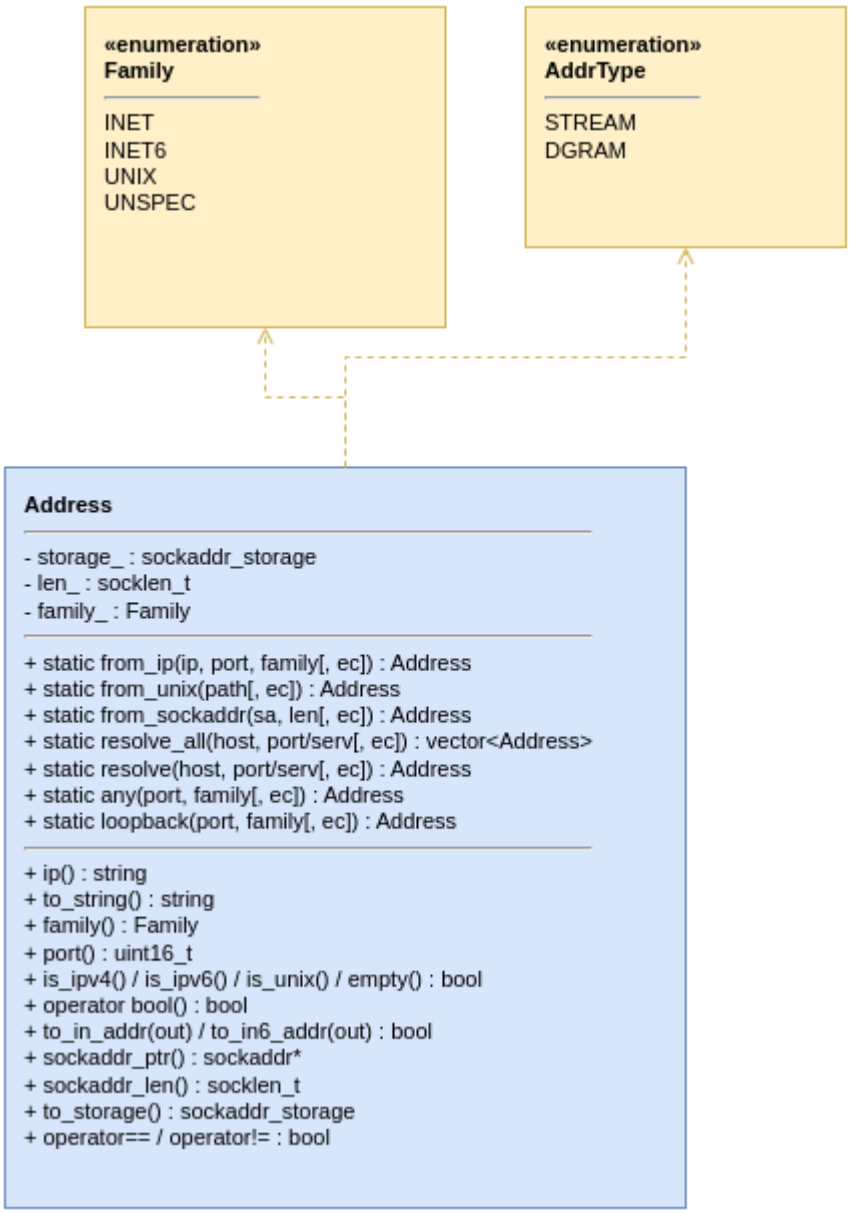
2.6 关键错误路径表

场景	返回	ec
`recv` 对端正常关闭	`0` 字节 / 空 vector	空
`recv` 出错	`0` / 空	`system_category(errno)`
`accept` 出错	`{Socket(), Address()}`	`system_category(errno)`
`resolve_all` 失败	空 vector	`gai_category(EAI_*)`
`resolve` 结果为空	`Address{}`	`gai_category(EAI_NONAME)`
`Selector::add` fd 已注册	—	`SelectorErrc::AlreadyRegistered`
`Selector::add` 负数 fd	—	`errc::invalid_argument`
`connect_timed` 超时	—	`errc::timed_out`
`getsockopt<T>` 长度不符	`T{}`	`errc::invalid_argument`
`Address::from_ip` 地址非法	`Address{}`	`errc::invalid_argument`
`Address::from_unix` 路径过长	`Address{}`	`errc::filename_too_long`
未初始化 epoll 后端操作	—	`EBADF`（`system_category`）

---

3. 地址模块设计（Address）

3.1 设计职责



`Address` 是 `struct sockaddr_storage` 的 RAII 值对象，承担：

- 统一承载 IPv4、IPv6、Unix Domain 三类地址；
- 提供强类型构造（`from_ip` / `from_unix` / `from_sockaddr`）；
- 提供 DNS 解析（`resolve` / `resolve_all`）；
- 提供常用地址快捷构造（`any` / `loopback`）；
- 提供字符串化（`ip` / `to_string`）；
- 提供与底层 API 互操作（`sockaddr_ptr` / `sockaddr_len` / `to_storage`）。

3.2 值语义

`Address` 采用**完整值语义**，可自由拷贝/赋值：

```
Address() = default;
Address(const Address& other);             // 逐字段 + storage_ 整体拷贝
Address& operator=(const Address& other); // 自赋值保护
~Address() = default;
```

拷贝时按成员逐一赋值（`storage_` 整体 `memcpy`、`len_`、`family_`），无需移动语义，因为其内部无资源所有权。

3.3 数据布局

```
class Address {
    struct sockaddr_storage storage_;   // 128 字节，覆盖所有 sockaddr_*
    socklen_t               len_;       // 实际长度
    Family                  family_;    // 冗余家族标记，快速判断
};
```

统一存储的动机：Linux 上 `sockaddr_storage` 足以容纳 `sockaddr_in`（16B）、`sockaddr_in6`（28B）、`sockaddr_un`（110B）。使用 `sockaddr_storage` 作为唯一载体，避免 `union` + tag 的手工管理。

`family_` 与 `storage_.ss_family` 冗余但保持一致，避免每次重新解释 `storage_` 的头字段；同时作为 `empty()` 判定依据。

内部 `clear()` 私有方法将三字段重置为初始状态（`storage_` 清零、`len_ = 0`、`family_ = UNSPEC`），所有 `*_impl` 内部实现在入口处调用它，保证构造失败时不残留脏数据。

3.4 枚举定义

```
enum class Family { INET, INET6, UNIX, UNSPEC };
enum class AddrType { STREAM, DGRAM };
```

`detail::to_af` / `from_af` / `to_socktype` 完成与系统常量（`AF_INET` 等）的双向映射，默认 `UNSPEC` 用于「未指定/自动判定」。

3.5 支持地址类型

家族	类型	构造入口	长度
`AF_INET`	TCP / UDP	`from_ip`, `resolve`, `any`, `loopback`	`sizeof(sockaddr_in)`
`AF_INET6`	TCP / UDP	同上	`sizeof(sockaddr_in6)`
`AF_UNIX`	流 / 数据报	`from_unix`	`SUN_LEN()`（含 `\0` 截断）
`AF_UNSPEC`	空	默认构造	0

3.6 构造 API 矩阵

静态函数	异常版	错误码版	说明
`from_ip(ip, port, family=UNSPEC)`	√	√	`inet_pton` 双栈自适应：先试 IPv4，失败再试 IPv6（`family==UNSPEC`）
`from_unix(path)`	√	√	路径长度 ≥ `sun_path` 容量时返回 `filename_too_long`
`from_sockaddr(sa, len)`	√	√	由 syscall 返回的 sockaddr 直接封装；`sa==nullptr` 或 `len` 超出 `storage_` 容量视为非法
`resolve_all(host, port/serv, family, type)`	√	√	`getaddrinfo`，返回全部候选
`resolve(host, port/serv, family, type)`	√	√	取 `resolve_all().front()`，空集 → `EAI_NONAME`
`any(port, family=INET)`	√	√	`INADDR_ANY` / `in6addr_any`；`UNIX` / `UNSPEC` 返回 `invalid_argument`
`loopback(port, family=INET)`	√	√	`INADDR_LOOPBACK` / `in6addr_loopback`

关键实现契约：

- `from_ip` 的 `family` 参数决定尝试顺序：`UNSPEC` 时先 IPv4 再 IPv6，指定族时只尝试对应族；两种尝试都失败返回 `invalid_argument`。
- 解析失败后 `ec` 被设为对应类别错误，返回的 `Address` 保持默认构造状态（`empty() == true`）。
- `resolve` 与 `resolve_all` 有两族重载：`uint16_t port` 版内部转字符串委托给 `std::string serv` 版，避免重复实现。

3.7 字符串化

方法	IPv4	IPv6	Unix
`ip()`	`inet_ntop`	`inet_ntop`	`sun_path`
`to_string()`	`1.2.3.4:80`	`[::1]:80`	`/tmp/x.sock`

IPv6 用 `[...]` 包裹，符合 RFC 3986 host 语法。未知族返回空串。`inet_ntop` 失败时 `ip()` 返回空串（不抛异常，作为查询接口保持轻量）。

3.8 属性查询

方法	语义
`family()`	返回内部 `Family` 标记
`port()`	IPv4/IPv6 返回 host 字节序端口；其他返回 0
`is_ipv4()` / `is_ipv6()` / `is_unix()`	快速族判定
`empty()`	`family_ == UNSPEC`
`operator bool()`	`!empty()`，便于 `if (addr)` 判断

3.9 比较与相等性

`operator==` 按族分别比较：

- 族不同 → 不等；
- IPv4：`sin_port` + `sin_addr.s_addr`；
- IPv6：`sin6_port` + `sin6_addr`（`memcmp`）；
- Unix：`memcmp` 整个 `sun_path` 定长数组（即使路径较短也按整段比较，语义上等价于「路径完全相同」）；
- 其他（`UNSPEC`）：视为相等。

`operator!=` 为 `!operator==`。

3.10 `to_in_addr` / `to_in6_addr`

仅当族匹配时输出并返回 `true`，否则返回 `false` 且**不修改**出参。此设计供上层组播等仅支持 IPv4 的接口安全调用。

3.11 底层访问

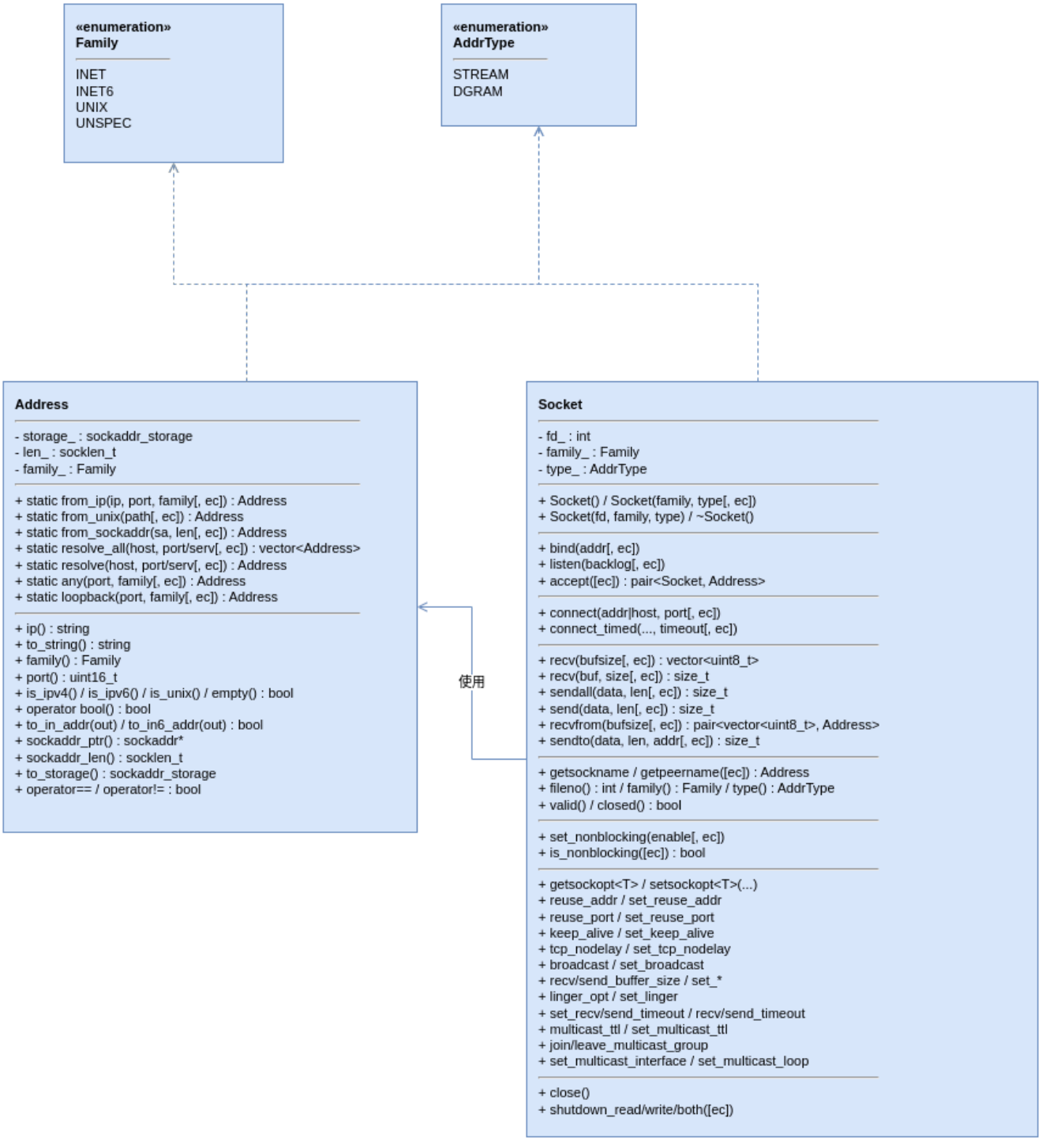
```
const struct sockaddr* sockaddr_ptr() const;
struct sockaddr*       sockaddr_ptr();
socklen_t              sockaddr_len() const;
struct sockaddr_storage to_storage() const;
```

`const` 与非 `const` 双版本：非 `const` 版本供用户侧构造场景（如自定义地址族扩展）使用。

---

4. Socket 模块设计（Socket）

4.1 类定义与生命周期




```
class Socket {
    int fd_ = -1;
    Family family_ = Family::UNSPEC;
    AddrType type_ = AddrType::STREAM;
};
```

```
Socket() noexcept;                                  // 空 socket（fd_ = -1）
Socket(Family, AddrType);                           // 异常版：创建即成功或抛
Socket(Family, AddrType, std::error_code&) noexcept; // 错误码版
Socket(int fd, Family, AddrType) noexcept;          // 接管 fd（供 accept 内部用）

Socket(const Socket&) = delete;                     // 禁止拷贝
Socket(Socket&&) noexcept;                          // 移动：源 fd 置 -1
Socket& operator=(Socket&&) noexcept;
~Socket();                                          // close() if fd_ >= 0
```

Rule of Five 策略：

操作	行为
默认构造	`fd_ = -1`，无效句柄
`Socket(Family, AddrType)`	创建底层 fd，失败抛 `SocketError`
`Socket(Family, AddrType, ec)`	创建底层 fd，失败置 `ec`；成功时同步写入 `family_`/`type_`
`Socket(int fd, Family, AddrType)`	内部使用（`accept` 产生），直接接管 fd
拷贝构造/赋值	**删除**，防止双重关闭
移动构造/赋值	接管 fd，源置 `-1` / `UNSPEC` / `STREAM`；移动赋值先 `close()` 自身
析构	`close()`，幂等

`valid()` / `operator bool()` 判断 `fd_ >= 0`。

4.2 服务端 API

方法	异常版	错误码版	备注
`bind(Address)`	√	√	直接绑定；异常版 `what` 中拼接 `addr.to_string()`
`listen(backlog=5)`	√	√	backlog 默认 5
`accept()` → `pair<Socket, Address>`	√	√	`EINTR` 重试；返回 peer 地址

accept 流程：

1. 循环调用 accept 系统调用，`EINTR` 时重试；
2. 失败 → `ec` 置位，返回 `{Socket{}, Address{}}`；
3. 成功 → 从 `sockaddr_storage` 构造 `Address`，返回 `{Socket(conn_fd, peer.family(), type_), peer}`。

返回值的 `Socket` 通过内部构造函数接管 fd，`type_` 沿用监听 socket 的类型（TCP/UDP），`family_` 取自 peer 地址。

设计说明：`accept` 保留阻塞语义，非阻塞场景由调用者配合 `Selector` 使用——先 `Selector::add(listen_fd, Read)`，等可读后再 `accept`。

4.3 客户端 API

方法	说明
`connect(Address)`	阻塞 connect，`EINTR` 重试；失败置 `ec`
`connect(host, port)`	内部 `Address::resolve(host, port, family_, type_, ec)` 再 connect；`ec` 可能来自 `gai_category` 或 `system_category`
`connect_timed(addr, timeout)`	**非阻塞 connect + select 超时**
`connect_timed(host, port, timeout)`	同上，带 DNS 解析

4.3.1 connect 语义说明

`connect` 可能返回 `EINPROGRESS`（非阻塞），本函数不特殊处理，原样透出，调用者需结合非阻塞语义判断。`EINTR` 时在循环内重试。

4.3.2 connect_timed 实现

非阻塞 connect + `select` 等待可写，是库中唯一独立实现的超时连接逻辑。

实现步骤：

1. **保存并设置非阻塞**：`fcntl(F_GETFL)` 取旧标志，`F_SETFL` 置 `O_NONBLOCK`；任一步失败立即透出 errno。
2. **恢复守卫**：局部结构体 `FlagsGuard` 持有 `{fd, old_flags}`，析构时 `F_SETFL` 恢复旧标志，保证任何返回路径（含异常）都不残留非阻塞状态。
3. **发起非阻塞 connect**：
   - 返回 0 → 立即成功（localhost 常见）；
   - `errno != EINPROGRESS` → 失败透出；
   - `EINPROGRESS` → 继续。
4. **select 等待**：读/写集合同时监听（连接失败时 fd 可读），`EINTR` 重试。
   - `n < 0` → errno 透出；
   - `n == 0` → `std::errc::timed_out`。
5. **SO_ERROR 取真实结果**：通过 `getsockopt<int>(SOL_SOCKET, SO_ERROR)` 读取，非 0 则作为错误透出。
6. 成功路径与失败路径均在离开函数时由守卫恢复原阻塞标志。

注意点：`struct timeval` 在 `select` 返回后可能被内核修改，故每次调用前必须重新构造（本实现通过 `to_timeval` 辅助函数每次新建）。恢复守卫保证语义一致。

便捷重载：`connect_timed(host, port, timeout, ec)` 先 `Address::resolve`，解析成功再委托给 `connect_timed(addr, ...)`；解析失败时直接返回，避免多余操作。

4.4 数据收发

方法	语义
`recv(size_t)` → `vector<uint8_t>`	分配 `bufsize` 临时 buffer，接收后 `resize` 到实际字节数
`recv(void*, size_t)` → `size_t`	零拷贝；返回字节数
`sendall(data)`	**循环 send 直到全部发出**，抑制 SIGPIPE
`send(data)`	单次 send，返回实际发送字节数（可能短写）
`recvfrom(bufsize)` → `pair<vector, Address>`	UDP，返回（数据，对端地址）
`sendto(data, Address)`	UDP，`EINTR` 重试

统一语义（recv 两重载）：

- `n == 0`：对端关闭，`ec` 为空，返回 0（或空 vector）；
- `n > 0`：正常；
- `n < 0`：`ec` 非空，返回 0（或空 vector）。

接收路径内部 `recv_impl` 处理 `EINTR` 重试。`recv(size_t)` 在 `bufsize == 0` 时返回 `invalid_argument`。

`sendall` 循环逻辑：

```
size_t sent = 0;
while (sent < len) {
    ssize_t n = send(剩余数据, MSG_NOSIGNAL);
    if (n < 0) {
        if (errno == EINTR) continue;
        ec = std::error_code(errno, std::system_category());
        return sent;   // 部分发送，调用方需检查 ec 与返回值
    }
    sent += n;
}
```

返回「已发送字节数」而非 bool，是为了在部分发送失败时调用者能准确知道进度（尤其 `sendall` 失败时）。发送路径统一使用 `MSG_NOSIGNAL` 抑制 SIGPIPE（对端关闭时进程不应被信号杀死）。

`send` 单次语义：`EINTR` 重试，返回本次实际发送字节数，可能短写；不做循环。

`recvfrom` / `sendto`（UDP）：

- `recvfrom` 使用栈上 `sockaddr_storage` 接收对端地址，成功后通过 `Address::from_sockaddr` 转换；
- `sendto` 通过 `Address` 的 `sockaddr_ptr()` / `sockaddr_len()` 直接使用，`EINTR` 重试。

4.5 查询 API

```
Address getsockname(std::error_code& ec) const noexcept;
Address getpeername(std::error_code& ec) const noexcept;
```

`const` 成员，因为不修改 socket 状态。使用栈上 `sockaddr_storage` + `socklen_t` 接收内核输出，成功后构造 `Address`。失败返回 `Address{}`。

4.6 非阻塞开关

`set_nonblocking(bool)` / `is_nonblocking()` 通过 `fcntl(F_GETFL / F_SETFL)` 操作 `O_NONBLOCK` 位。`is_nonblocking()` 在 `ec` 非空时返回 `false`，异常版抛 `SocketError`。`set_nonblocking` 默认参数 `enable = true`，便于直接调用「设为非阻塞」。

4.7 Socket Option 模板接口

```
template <typename T>
T getsockopt(int level, int optname, std::error_code& ec) const noexcept {
    ec.clear();
    T v{};
    socklen_t len = static_cast<socklen_t>(sizeof(T));
    getsockopt_impl(level, optname, &v, &len, ec);
    if (ec) return T{};
    if (len != sizeof(T)) {                    // 长度不符 → 类型选错了
        ec = std::make_error_code(std::errc::invalid_argument);
        return T{};
    }
    return v;
}

template <typename T>
void setsockopt(int level, int optname, const T& value, std::error_code& ec) noexcept;
```

- `T` 必须为 POD（`int`、`struct linger`、`struct timeval`、`unsigned char`、`struct in_addr`、`struct ip_mreq` 等）；
- `getsockopt<T>` 通过 `len` 出参校验：若内核返回的长度不等于 `sizeof(T)`，视为 `invalid_argument`，防止结构体版本不匹配导致的脏读；
- `setsockopt` 直接传 `&value` 与 `sizeof(T)`。

底层 `getsockopt_impl` / `setsockopt_impl` 提供异常版与错误码版，做空指针防御：`buf == nullptr` 或 `len == nullptr` 返回 `invalid_argument`，然后才调用系统接口。

4.8 Socket Option 便捷函数

每个 option 提供三件套（除纯写入 option）：

Option	读取	设置
`SO_REUSEADDR`	`reuse_addr()` / `reuse_addr(ec)`	`set_reuse_addr(bool)` / `set_reuse_addr(bool, ec)`
`SO_REUSEPORT`	同上	同上
`SO_KEEPALIVE`	同上	同上
`TCP_NODELAY`	同上（`IPPROTO_TCP`）	同上
`SO_BROADCAST`	同上	同上
`SO_RCVBUF`	`recv_buffer_size()` → int	`set_recv_buffer_size(int)`（非负校验）
`SO_SNDBUF`	`send_buffer_size()`	`set_send_buffer_size(int)`（非负校验）
`SO_LINGER`	`linger_opt()` → `struct linger`	`set_linger(bool on, int sec)`（sec 非负校验）
`SO_RCVTIMEO`	`recv_timeout()` → `milliseconds`	`set_recv_timeout(ms)`（ms 非负校验）
`SO_SNDTIMEO`	`send_timeout()`	`set_send_timeout(ms)`（ms 非负校验）

设计要点：

- 布尔选项统一用 `int` 0/1 传入内核（Linux 惯例）；读取时通过 `v != 0` 归一化为 `bool`；
- 时间选项在 `std::chrono::milliseconds` 与 `struct timeval` 之间转换（`to_timeval` / `to_milliseconds` 私有辅助），避免用户直接操作 `timeval`；
- 缓冲区大小 / linger 秒数 / 超时毫秒 `set_*` 校验非负，负值返回 `invalid_argument`，**不调用** `setsockopt`。

4.9 组播接口

```
int  multicast_ttl() const;
void set_multicast_ttl(int ttl);                        // 0..255
void join_multicast_group(const Address& group,
                          const Address& iface = Address{});
void leave_multicast_group(const Address& group,
                           const Address& iface = Address{});
void set_multicast_interface(const Address& iface);
void set_multicast_loop(bool on);
```

设计要点：

- 组播 TTL 校验 0~255；读取时以 `unsigned char` 类型 `getsockopt`，转换回 `int`；
- 组播加入/离开时，若 `iface` 为空则使用 `INADDR_ANY`；非 IPv4 地址通过 `Address::to_in_addr` 返回 false 时置 `invalid_argument`，**不进行** `setsockopt`；
- 组播加入/离开使用 `struct ip_mreq`，`imr_multiaddr` 来自 `group`，`imr_interface` 来自 `iface` 或 `INADDR_ANY`；

- `set_multicast_interface` 使用 `struct in_addr`（`IP_MULTICAST_IF` 接受 `in_addr` 而非 `ip_mreq`）；
- `set_multicast_loop` 参数以 `unsigned char` 传入内核。

4.10 关闭与半关闭

```
void close() noexcept;          // 幂等；Linux 上 close EINTR 不重试
void shutdown_write();          // SHUT_WR
void shutdown_read();           // SHUT_RD
void shutdown_both();           // SHUT_RDWR
bool closed() const noexcept;   // fd_ < 0
int  fileno() const noexcept;   // 裸 fd，供 Selector / 用户直接用
Family family() const noexcept;
AddrType type() const noexcept;
```

说明：`close()` 中**不重试** `EINTR`——Linux 上 `close` 返回 `EINTR` 时 fd 实际可能已被释放，重试有风险，这是 POSIX 的已知陷阱。`close()` 幂等，关闭后 `fd_ = -1`。

`shutdown_*` 三件套各提供异常版与错误码版，`EINTR` 场景同样原样透出（`shutdown` 不因 EINTR 导致资源状态不确定）。

`fileno()` 供 `Selector` 注册；`family()` / `type()` 供上层判断行为（如 `accept` 继承 `type_`）。

---

5. Selector 模块设计

5.1 Event 位掩码

```
enum class Event : uint32_t {
    None   = 0,
    Read   = 1 << 0,
    Write  = 1 << 1,
    Error  = 1 << 2,
    Hangup = 1 << 3,

    EdgeTrigger = 1u << 16,    // 模式位，不参与事件匹配
};
```

关键设计：`Event::EdgeTrigger` 放在 bit 16，与低 8 位的事件位隔离。原因是 `has_event(set, e)` 只应判断"事件"，不应把模式位当成事件。为此提供：

```
inline Event operator|(Event, Event);
inline Event operator&(Event, Event);
inline bool has_event(Event set, Event e);
inline Event event_bits(Event e);           // 屏蔽高 24 位，只保留低 8 位事件
inline bool  has_edge_trigger(Event e);     // 检测 ET
```

位域划分：

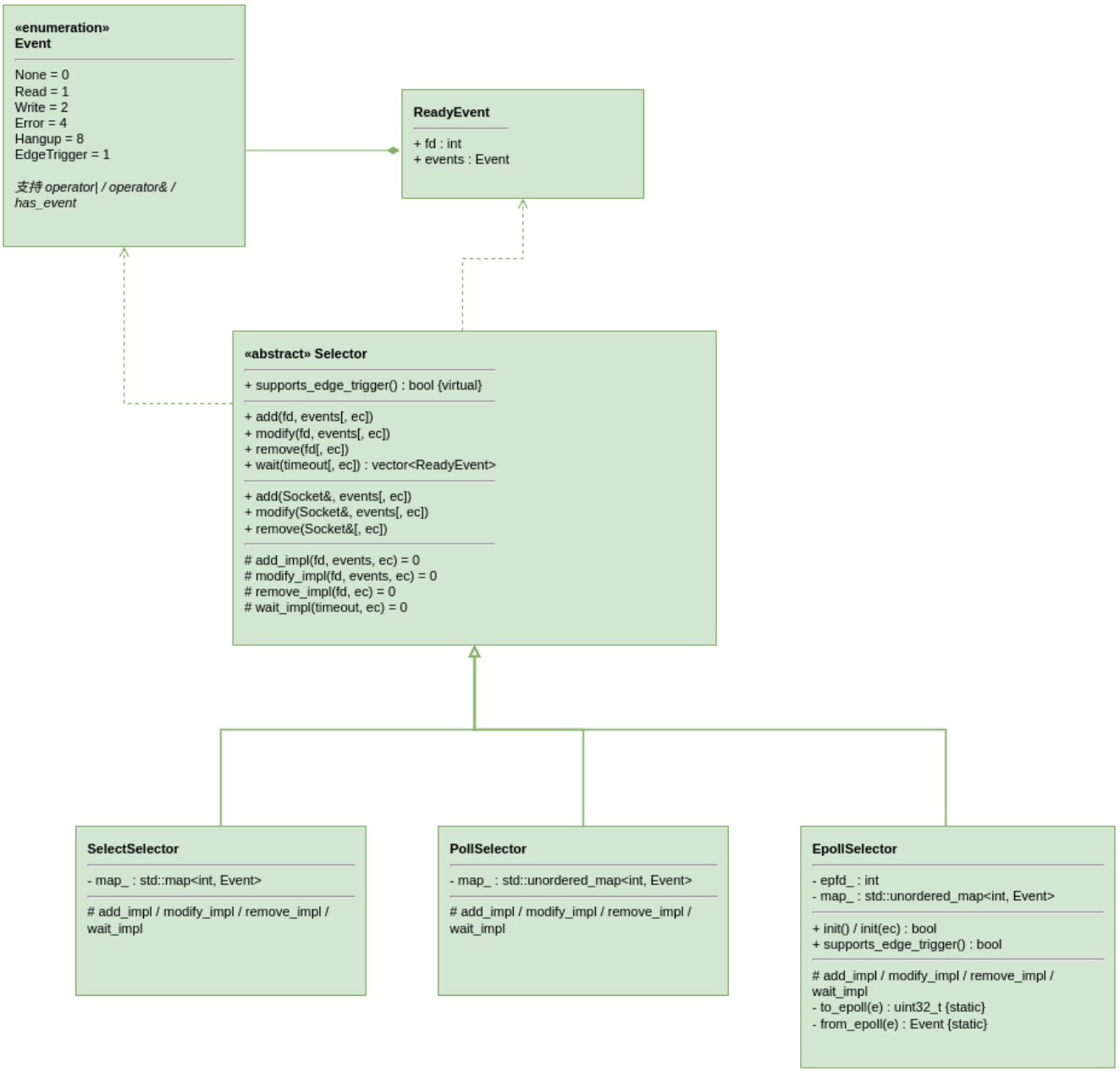
- 低 8 位（`0x00~0xFF`）：纯事件位，由内核报告；
- 高位（`1 << 16` 起）：模式位，仅作注册标志，**不会被报告**。

为什么 ET 也是 Event 的 bit：让用户可以用 `Event::Read | Event::EdgeTrigger` 一次表达 "读 + 边缘触发"，`Selector::add` 一个参数搞定，无需额外 API。语义上 ET 确实是 epoll 注册参数的一部分，作为 Event 的扩展位合理。

5.2 事件转换对照

unpsock	select	poll	epoll
`Read`	`readfds`	`POLLIN` / `POLLPRI`	`EPOLLIN` / `EPOLLPRI`
`Write`	`writefds`	`POLLOUT`	`EPOLLOUT`
`Error`	`exceptfds`	`POLLERR` / `POLLNVAL`	`EPOLLERR`
`Hangup`	—	`POLLHUP`	`EPOLLHUP`
`EdgeTrigger`	忽略	忽略	`EPOLLET`

5.3 Selector 抽象接口




```cpp
class Selector {
public:
    virtual ~Selector() = default;
    virtual bool supports_edge_trigger() const noexcept { return false; }

    // 四个公有封装（异常版 + 错误码版 × 3 个操作 + wait）
    void add(int fd, Event events);
    void add(int fd, Event events, std::error_code& ec);
    void modify(int fd, Event events);
    void modify(int fd, Event events, std::error_code& ec);
    void remove(int fd);
    void remove(int fd, std::error_code& ec);

    std::vector<ReadyEvent> wait(std::optional<std::chrono::milliseconds> = std::nullopt);
```

```
    std::vector<ReadyEvent> wait(std::optional<std::chrono::milliseconds>, std::error_code&
ec);

    // Socket 重载（转 fileno() 后调上面的）
    void add(const Socket&, Event);
    void add(const Socket&, Event, std::error_code&);
    void modify(const Socket&, Event);
    void modify(const Socket&, Event, std::error_code&);
    void remove(const Socket&);
    void remove(const Socket&, std::error_code&);

private:
    virtual void add_impl(int, Event, std::error_code&) = 0;
    virtual void modify_impl(int, Event, std::error_code&) = 0;
    virtual void remove_impl(int, std::error_code&) = 0;
    virtual std::vector<ReadyEvent> wait_impl(std::optional<std::chrono::milliseconds>,
                                              std::error_code&) = 0;
};
```

NVI 模式：公有非虚接口负责异常/错误码双风格转换，私有虚接口是纯错误码版——子类实现只需写一份逻辑。`add(const Socket&)` 重载免去调用者手动 `fileno()`。

`supports_edge_trigger()`：供上层查询后端能力，默认 `false`（select/poll 不支持 ET），epoll 覆写为 `true`。

`wait` 的超时语义：

- `std::nullopt` → 永久阻塞（对应 `-1`）；
- `0ms` → 非阻塞轮询；
- `> 0` → 超时返回空 vector（`ec` 为空，与出错区分）。

NVI 实现细节：异常版 `add` / `modify` / `remove` 内部先 `std::error_code ec` 调用错误码版，若 `ec` 非空则抛出 `SelectorError(ec, "...")`，`what` 消息前缀按操作区分（`"selector add failed"` 等）。

5.4 ReadyEvent

```
struct ReadyEvent {
    int   fd;
    Event events;   // 只含 Read/Write/Error/Hangup，不含 EdgeTrigger
};
```

`wait()` 返回 `vector<ReadyEvent>`，调用方按顺序处理。不做 fd → 回调的分发，保持库的"零框架"定位。只包含实际就绪的 fd，未就绪的不出现，简化调用者遍历。

事件归位：后端 `from_*` 转换只写入事件位，不写 `EdgeTrigger`——ET 是注册期语义，运行时无意义。

5.5 后端选择策略

库不强制用户用哪个后端，但提供工厂契约：

```
std::unique_ptr<Selector> make_select_selector();
std::unique_ptr<Selector> make_select_selector(std::error_code&);
std::unique_ptr<Selector> make_poll_selector();
std::unique_ptr<Selector> make_poll_selector(std::error_code&);
std::unique_ptr<Selector> make_epoll_selector();
std::unique_ptr<Selector> make_epoll_selector(std::error_code&);
```

5.6 select 后端

数据结构：`std::map<int, Event>`（有序，便于遍历时取最大 fd）。实现类禁止拷贝与移动。

注册/修改/移除：仅操作内存映射，`wait` 时动态构造 fd 集合。重复注册返回 `AlreadyRegistered`；修改/移除未注册 fd 返回 `NotRegistered`。

wait 实现：

1. 遍历映射，按事件位 `FD_SET` 到 `readfds` / `writefds`；`exceptfds` **总是**把所有 fd 加入（select 的异常集合对应错误/带外数据）；
2. `timeout` 转换为 `struct timeval`（毫秒 → 秒+微秒）；`nullopt` 传 `nullptr` 表示无限等待；
3. 执行 `::select`，`EINTR` 重试；
4. `n < 0` → 置 `ec`；`n == 0` → 空 vector（超时）；
5. 遍历映射，`FD_ISSET` 命中读/写集合映射为 `Read`/`Write`，命中异常集合映射为 `Error`；`ready != None` 才加入结果。

5.7 poll 后端

数据结构：`std::unordered_map<int, Event>`（`wait` 时整体构造 `pollfd` 数组）。实现类禁止拷贝与移动。

wait 实现：

1. 遍历映射，构造 `std::vector<struct pollfd>`，`events` 按事件位设置（`Read` → `POLLIN`，`Write` → `POLLOUT`）；
2. `timeout_ms = timeout ? static_cast<int>(count()) : -1`（负值取 0）；
3. 执行 `::poll`，`EINTR` 重试；
4. `n < 0` → 置 `ec`；`n == 0` → 空 vector（超时）；
5. 遍历返回的 `pollfd`，`revents == 0` 跳过，否则 `from_poll` 转换；`None` 跳过。

事件映射（接收侧）：

内核 `revents`	`Event`
`POLLIN` / `POLLPRI`	`Read`
`POLLOUT`	`Write`
`POLLERR` / `POLLNVAL`	`Error`
`POLLHUP`	`Hangup`

`POLLERR` / `POLLHUP` / `POLLNVAL` 无需显式请求，`poll` 总会报告。

5.8 epoll 后端

数据结构：内核 epoll 实例 fd（`epoll_create1(EPOLL_CLOEXEC)` 创建）+ `std::unordered_map<int, Event>`（记录注册事件）。实现类禁止拷贝与赋值；析构时 `::close(epfd_)`（若已创建）。

初始化双模式：

- `init()`：失败抛 `SelectorError(ec, "init failed")`；
- `init(ec)`：失败返回 false 并置 `ec`；已初始化时直接返回 true（幂等）；
- 构造后为「未初始化」状态（`epfd_ == -1`）；
- 所有操作入口经 `check_init` 检查，未初始化返回 `EBADF`（`system_category`）。

工厂函数：`make_epoll_selector(ec)` 内部构造 `EpollSelector` 并调用 `init(ec)`；失败返回 `nullptr`。这是 epoll 与 select/poll 工厂的唯一差异（epoll 可能初始化失败）。

add 实现：

1. `check_init`，失败返回；
2. 校验 `map_.count(fd) == 0`，否则 `AlreadyRegistered`；
3. 构造 `struct epoll_event{events = to_epoll(events), data.fd = fd}`；
4. 调用 `::epoll_ctl(EPOLL_CTL_ADD, ...)`，失败置 `ec`（`system_category`）；
5. 成功后 `map_[fd] = events`。

modify 实现：校验初始化、已注册（否则 `NotRegistered`）→ `EPOLL_CTL_MOD` → 成功后更新 `map_`。

remove 实现：校验初始化、已注册 → 调用 `EPOLL_CTL_DEL` → 成功后 `map_.erase(it)`。

wait 实现：

1. `check_init`；
2. 计算 `timeout_ms`：`nullopt → -1`；`count() <= 0 → 0`；否则强转 `int`；
3. 栈上 `struct epoll_event evs[MAX_EVENTS]`，`MAX_EVENTS = 64`；
4. `::epoll_wait` 调用，`EINTR` 重试；
5. `n < 0` → 置 `ec`；`n == 0` → 空 vector（超时）；
6. 遍历 `evs[0..n)`，`{data.fd, from_epoll(events)}` 构造 `ReadyEvent`；`reserve(n)` 避免多次分配。

事件映射：

注册方向	`Event`	`epoll`
写入	`Read`	`EPOLLIN`
写入	`Write`	`EPOLLOUT`
写入	`EdgeTrigger`（模式位）	`EPOLLET`

接收方向	`epoll`	`Event`
读取	`EPOLLIN` / `EPOLLPRI`	`Read`
读取	`EPOLLOUT`	`Write`
读取	`EPOLLERR`	`Error`
读取	`EPOLLHUP`	`Hangup`

`to_epoll` 通过 `has_edge_trigger(e)` 决定是否附加 `EPOLLET`——ET 是注册期语义，`from_epoll` 不会输出 ET。`EPOLLERR` / `EPOLLHUP` 内核总会报告，无需显式请求。

`supports_edge_trigger()` 返回 `true`。

---

6. 主机名模块

```
namespace host {
    inline std::string local_hostname();
}
```

调用系统主机名查询接口（`gethostname`），缓冲区 256 字节，末尾强制截断为 `\0` 防御性处理。失败返回空串（非关键路径，不抛异常）。

设计取舍：主机名查询失败极少见且通常无补救手段，返回空串比抛异常更友好；调用者可用 `empty()` 判断。

---

7. 典型使用场景

7.1 TCP 服务端（epoll ET）

```
unpsock::Socket listen_sock(unpsock::Family::INET, unpsock::AddrType::STREAM);
listen_sock.set_reuse_addr(true);
listen_sock.bind(unpsock::Address::any(8080));
listen_sock.listen(128);
listen_sock.set_nonblocking(true);

auto sel = unpsock::make_epoll_selector(ec);
sel->add(listen_sock, unpsock::Event::Read | unpsock::Event::EdgeTrigger);

while (running) {
    for (auto& ev : sel->wait(std::chrono::milliseconds(100))) {
        if (ev.fd == listen_sock.fileno()) {
            auto [conn, peer] = listen_sock.accept(ec);
            if (!ec) {
                conn.set_nonblocking(true);
                sel->add(conn, unpsock::Event::Read | unpsock::Event::EdgeTrigger);
            }
        }
    }
```

```
}
```

7.2 TCP 客户端（带超时）

```
unpsock::Socket sock(unpsock::Family::INET, unpsock::AddrType::STREAM);
sock.connect_timed("example.com", 80, std::chrono::seconds(3), ec);
if (!ec) sock.sendall("GET / HTTP/1.1\r\n\r\n");
```

7.3 UDP 组播接收

```
unpsock::Socket sock(unpsock::Family::INET, unpsock::AddrType::DGRAM);
sock.set_reuse_addr(true);
sock.bind(unpsock::Address::any(5353));
sock.join_multicast_group(unpsock::Address::from_ip("239.255.0.1", 5353));
auto [data, from] = sock.recvfrom(65535, ec);
```

7.4 双风格错误处理对比

```
// 异常版：原型验证，逻辑直白
try {
    unpsock::Socket sock(unpsock::Family::INET, unpsock::AddrType::STREAM);
    sock.connect("192.168.1.10", 8080);
    sock.sendall(data);
} catch (const unpsock::SocketException& e) {
    log("socket error: %d %s", e.code().value(), e.what());
}

// 错误码版：生产代码，避免异常开销
unpsock::Socket sock(unpsock::Family::INET, unpsock::AddrType::STREAM, ec);
if (!ec) sock.connect("192.168.1.10", 8080, ec);
if (!ec) sock.sendall(data, ec);
if (ec) log("socket error: %d %s", ec.value(), ec.message().c_str());
```